

In [2]: *# CELL 1: Environment Setup & Library Imports*

```
import os, cv2, warnings, random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
from tqdm import tqdm
warnings.filterwarnings('ignore')

# ML
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (classification_report, confusion_matrix,
                             roc_curve, auc, ConfusionMatrixDisplay)
from sklearn.utils import resample

# Deep Learning
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models, regularizers, callbacks
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import MobileNetV2, ResNet50, EfficientNetB0

# Reproducibility – critical for IEEE
SEED = 42
os.environ['PYTHONHASHSEED'] = str(SEED)
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

# GPU check
print("="*55)
print(f"\n TensorFlow version : {tf.__version__}")
gpus = tf.config.list_physical_devices('GPU')
print(f" GPU available      : {'Yes – ' + gpus[0].name if gpus else 'No (CPU)'}")
print(f" Random seed fixed   : {SEED}")
print("\n All libraries loaded successfully.")
print("="*55)
```

```

2026-03-24 13:50:10.219682: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register cuFFT factory: Attempting to register factory for plugin cuFFT when one has already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
E0000 00:00:1774360210.583504      55 cuda_dnn.cc:8579] Unable to register cuDNN factory: Attempting to register factory for plugin cuDNN when one has already been registered
E0000 00:00:1774360210.686610      55 cuda_blas.cc:1407] Unable to register cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has already been registered
W0000 00:00:1774360211.562994      55 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.
W0000 00:00:1774360211.563039      55 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.
W0000 00:00:1774360211.563042      55 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.
W0000 00:00:1774360211.563044      55 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.

```

```

=====

TensorFlow version : 2.19.0
GPU available      : Yes - /physical_device:GPU:0
Random seed fixed  : 42

All libraries loaded successfully.
=====

```

In [3]: *# CELL 2: Dataset Scan & Label Extraction*

```

BASE_PATH = '/kaggle/input/datasets/ruizgara/socofing/SOC0Fing/Real'
IMG_SIZE = 128

images = []
labels_gender = []
labels_finger = []
labels_hand = []

def extract_labels(filename):
    name = filename.replace('.BMP', '').replace('.bmp', '')
    parts = name.split('__')
    meta = parts[1].split('_')
    gender = meta[0]          # M or F
    hand = meta[1]           # Left or Right
    finger = meta[2] + '_' + meta[3] # e.g. little_finger
    return gender, hand, finger

for fname in tqdm(sorted(os.listdir(BASE_PATH)), desc="Loading"):
    if not (fname.endswith('.BMP') or fname.endswith('.bmp')):

```

```

        continue
    try:
        gender, hand, finger = extract_labels(fname)
        img = cv2.imread(os.path.join(BASE_PATH, fname), cv2.IMREAD_GRAYSCALE)
        img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
        images.append(img)
        labels_gender.append(gender)
        labels_hand.append(hand)
        labels_finger.append(finger)
    except:
        continue

images = np.array(images)
labels_gender = np.array(labels_gender)

print(f"Total images loaded : {len(images)}")
print(f"Image shape          : {images[0].shape}")
print(f"\nGender distribution:")
unique, counts = np.unique(labels_gender, return_counts=True)
for u, c in zip(unique, counts):
    print(f"  {u} : {c} ({c/len(labels_gender)*100:.1f}%)")

```

Loading: 100%|██████████| 6000/6000 [01:21<00:00, 73.36it/s]

Total images loaded : 6000

Image shape : (128, 128)

Gender distribution:

F : 1230 (20.5%)

M : 4770 (79.5%)

In [8]: *# CELL 3 (RED0): Correct Preprocessing – No Oversampling, Use Class Weights*

```

from tensorflow.keras.utils import to_categorical

IMG_SIZE = 224

# Reload at 224x224
images_224 = []
labels_g    = []

for fname in tqdm(sorted(os.listdir(BASE_PATH)), desc="Loading 224"):
    if not (fname.endswith('.BMP') or fname.endswith('.bmp')):
        continue
    try:
        gender, hand, finger = extract_labels(fname)
        img = cv2.imread(os.path.join(BASE_PATH, fname), cv2.IMREAD_GRAYSCALE)
        img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
        images_224.append(img)
        labels_g.append(gender)
    except:
        continue

X_all = np.array(images_224).astype('float32') / 255.0

```

```

X_all = X_all.reshape(-1, IMG_SIZE, IMG_SIZE, 1)
y_all = np.array(labels_g)

le = LabelEncoder()
y_enc = le.fit_transform(y_all) # F=0, M=1
y_cat = to_categorical(y_enc, num_classes=2)

print(f"Class mapping : {dict(zip(le.classes_, le.transform(le.classes_)))}")
print(f"Total samples : {len(X_all)}")
unique, counts = np.unique(y_enc, return_counts=True)
for u, c in zip(unique, counts):
    print(f" {le.classes_[u]} : {c}")

# Stratified split on original data
X_tv, X_test, y_tv, y_test, y_tv_enc_, y_test_enc_ = train_test_split(
    X_all, y_cat, y_enc,
    test_size=0.15, random_state=SEED, stratify=y_enc)

y_tv_enc = np.argmax(y_tv, axis=1)
X_train, X_val, y_train, y_val = train_test_split(
    X_tv, y_tv,
    test_size=0.176, random_state=SEED, stratify=y_tv_enc)

print(f"\nTrain : {len(X_train)}")
print(f"Val   : {len(X_val)}")
print(f"Test  : {len(X_test)}")

# Class weights to handle imbalance – no duplication needed
from sklearn.utils.class_weight import compute_class_weight
cw = compute_class_weight('balanced', classes=np.unique(y_enc), y=y_enc)
class_weights = {0: cw[0], 1: cw[1]}
print(f"\nClass weights : {class_weights}")

```

Loading 224: 100%|██████████| 6000/6000 [00:07<00:00, 796.42it/s]

Class mapping : {np.str_('F'): np.int64(0), np.str_('M'): np.int64(1)}

Total samples : 6000

F : 1230

M : 4770

Train : 4202

Val : 898

Test : 900

Class weights : {0: np.float64(2.4390243902439024), 1: np.float64(0.6289308176100629)}

In [9]: # CELL 4: Model 1 – EfficientNetB0 Transfer Learning

```

from tensorflow.keras.applications import EfficientNetB0

def build_efficientnet(input_shape=(224, 224, 1), num_classes=2):
    inp = layers.Input(shape=input_shape)
    # grayscale to 3-channel
    x = layers.Conv2D(3, (1,1), padding='same')(inp)

```

```

x      = layers.BatchNormalization()(x)

base = EfficientNetB0(include_top=False, weights='imagenet',
                      input_shape=(224, 224, 3))
base.trainable = False

x      = base(x, training=False)
x      = layers.GlobalAveragePooling2D()(x)
x      = layers.Dense(256, activation='relu',
                      kernel_regularizer=regularizers.l2(1e-4))(x)
x      = layers.BatchNormalization()(x)
x      = layers.Dropout(0.5)(x)
x      = layers.Dense(128, activation='relu',
                      kernel_regularizer=regularizers.l2(1e-4))(x)
x      = layers.Dropout(0.3)(x)
out    = layers.Dense(num_classes, activation='softmax')(x)

return models.Model(inp, out)

aug = ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.10,
    height_shift_range=0.10,
    zoom_range=0.15,
    horizontal_flip=True,
    fill_mode='nearest'
)

eff_model = build_efficientnet()

# Phase 1 - train head only
eff_model.compile(optimizer=Adam(1e-3),
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

print(f"Trainable params (Phase 1): {sum([tf.size(w).numpy() for w in eff_model.trainable_weights])}")
print("Phase 1: Training head...")

h1 = eff_model.fit(
    aug.flow(X_train, y_train, batch_size=32),
    validation_data=(X_val, y_val),
    epochs=25,
    class_weight=class_weights,
    callbacks=[
        callbacks.EarlyStopping(monitor='val_accuracy', patience=8,
                                restore_best_weights=True, verbose=1),
        callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5,
                                    patience=4, min_lr=1e-7, verbose=1),
        callbacks.ModelCheckpoint('model_efficientnet.h5', monitor='val_accuracy',
                                  save_best_only=True, verbose=0)
    ],
    verbose=1
)

```

```

# Phase 2 – unfreeze top 50 layers
print("\nPhase 2: Fine-tuning...")
for layer in eff_model.layers:
    if hasattr(layer, 'layers'):
        for l in layer.layers[-50:]:
            if not isinstance(l, layers.BatchNormalization):
                l.trainable = True

eff_model.compile(optimizer=Adam(2e-5),
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

print(f"Trainable params (Phase 2): {sum([tf.size(w).numpy() for w in eff_model.get_weights()])}")

h2 = eff_model.fit(
    aug.flow(X_train, y_train, batch_size=16),
    validation_data=(X_val, y_val),
    epochs=40,
    class_weight=class_weights,
    callbacks=[
        callbacks.EarlyStopping(monitor='val_accuracy', patience=10,
                                restore_best_weights=True, verbose=1),
        callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.3,
                                     patience=5, min_lr=1e-8, verbose=1),
        callbacks.ModelCheckpoint('model_efficientnet.h5', monitor='val_accuracy',
                                   save_best_only=True, verbose=0)
    ],
    verbose=1
)

loss, acc = eff_model.evaluate(X_test, y_test, verbose=0)
print(f"\nEfficientNetB0 – Test Accuracy : {acc*100:.2f}%")
print(f"EfficientNetB0 – Test Loss      : {loss:.4f}")

```

Downloading data from https://storage.googleapis.com/keras-applications/efficientnetb0_notop.h5

16705208/16705208 ————— 1s 0us/step

Trainable params (Phase 1): 361,614

Phase 1: Training head...

Epoch 1/25

132/132 ————— 0s 243ms/step - accuracy: 0.5441 - loss: 0.8981

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

132/132 ————— 76s 351ms/step - accuracy: 0.5441 - loss: 0.8978 - val_accuracy: 0.2049 - val_loss: 0.8923 - learning_rate: 0.0010

Epoch 2/25

132/132 ————— 0s 107ms/step - accuracy: 0.5433 - loss: 0.8140

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

132/132  **15s** 115ms/step - accuracy: 0.5435 - loss: 0.8138 - val_accuracy: 0.7951 - val_loss: 0.6751 - learning_rate: 0.0010
Epoch 3/25

132/132  **15s** 111ms/step - accuracy: 0.5857 - loss: 0.7573 - val_accuracy: 0.7840 - val_loss: 0.6912 - learning_rate: 0.0010
Epoch 4/25

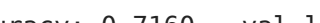
132/132  **15s** 110ms/step - accuracy: 0.5864 - loss: 0.7329 - val_accuracy: 0.4454 - val_loss: 0.8613 - learning_rate: 0.0010
Epoch 5/25

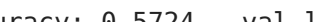
132/132  **15s** 110ms/step - accuracy: 0.5535 - loss: 0.7303 - val_accuracy: 0.4254 - val_loss: 0.8770 - learning_rate: 0.0010
Epoch 6/25

132/132  **15s** 111ms/step - accuracy: 0.6121 - loss: 0.7188 - val_accuracy: 0.7829 - val_loss: 0.6004 - learning_rate: 0.0010
Epoch 7/25

132/132  **15s** 111ms/step - accuracy: 0.5668 - loss: 0.7065 - val_accuracy: 0.6392 - val_loss: 0.6929 - learning_rate: 0.0010
Epoch 8/25

132/132  **14s** 108ms/step - accuracy: 0.5663 - loss: 0.6977 - val_accuracy: 0.7071 - val_loss: 0.6113 - learning_rate: 0.0010
Epoch 9/25

132/132  **15s** 111ms/step - accuracy: 0.5930 - loss: 0.6956 - val_accuracy: 0.7160 - val_loss: 0.5804 - learning_rate: 0.0010
Epoch 10/25

132/132  **15s** 110ms/step - accuracy: 0.5612 - loss: 0.7067 - val_accuracy: 0.5724 - val_loss: 0.7023 - learning_rate: 0.0010
Epoch 10: early stopping
Restoring model weights from the end of the best epoch: 2.

Phase 2: Fine-tuning...

Trainable params (Phase 2): 2,870,302

Epoch 1/40

2026-03-24 14:19:51.393165: E external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay kernel timed out: measured time has sub-optimal accuracy. There may be a missing warmup execution, please investigate in Nsight Systems.
2026-03-24 14:19:51.598586: E external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay kernel timed out: measured time has sub-optimal accuracy. There may be a missing warmup execution, please investigate in Nsight Systems.
2026-03-24 14:19:51.913976: E external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay kernel timed out: measured time has sub-optimal accuracy. There may be a missing warmup execution, please investigate in Nsight Systems.
2026-03-24 14:19:52.119841: E external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay kernel timed out: measured time has sub-optimal accuracy. There may be a missing warmup execution, please investigate in Nsight Systems.

22/263  **13s** 56ms/step - accuracy: 0.6313 - loss: 0.8453

2026-03-24 14:20:09.547749: E external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay kernel timed out: measured time has sub-optimal accuracy. There may be a missing warmup execution, please investigate in Nsight Systems.
2026-03-24 14:20:09.752933: E external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay kernel timed out: measured time has sub-optimal accuracy. There may be a missing warmup execution, please investigate in Nsight Systems.
2026-03-24 14:20:10.068123: E external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay kernel timed out: measured time has sub-optimal accuracy. There may be a missing warmup execution, please investigate in Nsight Systems.
2026-03-24 14:20:10.273353: E external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay kernel timed out: measured time has sub-optimal accuracy. There may be a missing warmup execution, please investigate in Nsight Systems.

263/263 ————— **0s** 119ms/step - accuracy: 0.6190 - loss: 0.8031

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

263/263 ————— **78s** 166ms/step - accuracy: 0.6189 - loss: 0.8030 - val_accuracy: 0.7004 - val_loss: 0.6539 - learning_rate: 2.0000e-05

Epoch 2/40

263/263 ————— **15s** 58ms/step - accuracy: 0.5862 - loss: 0.7536 - val_accuracy: 0.5813 - val_loss: 0.6823 - learning_rate: 2.0000e-05

Epoch 3/40

263/263 ————— **15s** 58ms/step - accuracy: 0.5869 - loss: 0.7353 - val_accuracy: 0.6771 - val_loss: 0.6392 - learning_rate: 2.0000e-05

Epoch 4/40

263/263 ————— **0s** 55ms/step - accuracy: 0.5560 - loss: 0.7479

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

263/263 ————— **16s** 59ms/step - accuracy: 0.5561 - loss: 0.7479 - val_accuracy: 0.7027 - val_loss: 0.5843 - learning_rate: 2.0000e-05

Epoch 5/40

263/263 ————— **0s** 54ms/step - accuracy: 0.5696 - loss: 0.7290

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

263/263 ————— **15s** 59ms/step - accuracy: 0.5696 - loss: 0.7290 - val_accuracy: 0.7372 - val_loss: 0.5967 - learning_rate: 2.0000e-05

Epoch 6/40

263/263 ————— **0s** 55ms/step - accuracy: 0.5796 - loss: 0.7245

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

263/263 ————— 16s 59ms/step - accuracy: 0.5796 - loss: 0.7244 -
val_accuracy: 0.7528 - val_loss: 0.5454 - learning_rate: 2.0000e-05
Epoch 7/40

263/263 ————— 15s 58ms/step - accuracy: 0.5962 - loss: 0.7087 -
val_accuracy: 0.6592 - val_loss: 0.6013 - learning_rate: 2.0000e-05
Epoch 8/40

263/263 ————— 15s 58ms/step - accuracy: 0.5760 - loss: 0.7067 -
val_accuracy: 0.5746 - val_loss: 0.7040 - learning_rate: 2.0000e-05
Epoch 9/40

263/263 ————— 15s 57ms/step - accuracy: 0.5654 - loss: 0.7230 -
val_accuracy: 0.6325 - val_loss: 0.6516 - learning_rate: 2.0000e-05
Epoch 10/40

263/263 ————— 15s 57ms/step - accuracy: 0.5873 - loss: 0.7129 -
val_accuracy: 0.6281 - val_loss: 0.6467 - learning_rate: 2.0000e-05
Epoch 11/40

263/263 ————— 0s 54ms/step - accuracy: 0.5810 - loss: 0.7326
Epoch 11: ReduceLRonPlateau reducing learning rate to 5.999999848427251e-06.

263/263 ————— 15s 57ms/step - accuracy: 0.5810 - loss: 0.7325 -
val_accuracy: 0.6169 - val_loss: 0.6743 - learning_rate: 2.0000e-05
Epoch 12/40

263/263 ————— 15s 58ms/step - accuracy: 0.5954 - loss: 0.6932 -
val_accuracy: 0.6325 - val_loss: 0.6539 - learning_rate: 6.0000e-06
Epoch 13/40

263/263 ————— 16s 60ms/step - accuracy: 0.5759 - loss: 0.7149 -
val_accuracy: 0.6581 - val_loss: 0.6268 - learning_rate: 6.0000e-06
Epoch 14/40

263/263 ————— 16s 60ms/step - accuracy: 0.5908 - loss: 0.7133 -
val_accuracy: 0.6693 - val_loss: 0.6067 - learning_rate: 6.0000e-06
Epoch 15/40

263/263 ————— 16s 60ms/step - accuracy: 0.5935 - loss: 0.7068 -
val_accuracy: 0.6192 - val_loss: 0.6643 - learning_rate: 6.0000e-06
Epoch 16/40

263/263 ————— 0s 57ms/step - accuracy: 0.5933 - loss: 0.7158
Epoch 16: ReduceLRonPlateau reducing learning rate to 1.7999999272433341e-06.

263/263 ————— 16s 60ms/step - accuracy: 0.5933 - loss: 0.7157 -
val_accuracy: 0.6303 - val_loss: 0.6351 - learning_rate: 6.0000e-06
Epoch 16: early stopping
Restoring model weights from the end of the best epoch: 6.

EfficientNetB0 – Test Accuracy : 75.00%
EfficientNetB0 – Test Loss : 0.5539

In [10]: # CELL 5: Diagnosis – what is the model actually predicting?

```
y_pred_probs = eff_model.predict(X_test, verbose=0)
y_pred       = np.argmax(y_pred_probs, axis=1)
y_true       = np.argmax(y_test, axis=1)

print("Confusion Matrix:")
cm = confusion_matrix(y_true, y_pred)
print(cm)

print("\nClassification Report:")
print(classification_report(y_true, y_pred, target_names=le.classes_))
```

```

print(f"\nPredicted F : {np.sum(y_pred == 0)}")
print(f"Predicted M : {np.sum(y_pred == 1)}")
print(f"True F      : {np.sum(y_true == 0)}")
print(f"True M      : {np.sum(y_true == 1)}")

# Check prediction confidence
print(f"\nMean confidence F predictions : {y_pred_probs[y_pred==0, 0].mean():.3f}")
print(f"Mean confidence M predictions : {y_pred_probs[y_pred==1, 1].mean():.3f}")

```

Confusion Matrix:

```

[[ 49 136]
 [ 89 626]]

```

Classification Report:

	precision	recall	f1-score	support
F	0.36	0.26	0.30	185
M	0.82	0.88	0.85	715
accuracy			0.75	900
macro avg	0.59	0.57	0.58	900
weighted avg	0.73	0.75	0.74	900

```

Predicted F : 138
Predicted M : 762
True F      : 185
True M      : 715

```

Mean confidence F predictions : 0.572

Mean confidence M predictions : 0.713

In [11]: *# CELL 6: CLAHE Preprocessing + Balanced Batch Generator*

```

import cv2
from tensorflow.keras.utils import Sequence

# Apply CLAHE to enhance fingerprint ridges
def apply_clahe(images_array):
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
    result = []
    for img in tqdm(images_array, desc="CLAHE"):
        img_u8 = (img.reshape(IMG_SIZE, IMG_SIZE) * 255).astype(np.uint8)
        enhanced = clahe.apply(img_u8)
        result.append(enhanced.astype('float32') / 255.0)
    return np.array(result).reshape(-1, IMG_SIZE, IMG_SIZE, 1)

print("Applying CLAHE enhancement...")
X_train_c = apply_clahe(X_train)
X_val_c   = apply_clahe(X_val)
X_test_c  = apply_clahe(X_test)

# Verify

```

```

y_train_enc = np.argmax(y_train, axis=1)
y_val_enc   = np.argmax(y_val,   axis=1)
y_test_enc  = np.argmax(y_test,  axis=1)

idx_F_train = np.where(y_train_enc == 0)[0]
idx_M_train = np.where(y_train_enc == 1)[0]
print(f"Train - F: {len(idx_F_train)} | M: {len(idx_M_train)}")

# Balanced batch generator - equal F and M per batch
class BalancedGenerator(Sequence):
    def __init__(self, X, y, batch_size=32, augment=True):
        self.X          = X
        self.y           = y
        self.batch_size  = batch_size
        self.augment     = augment
        self.idx_F       = np.where(np.argmax(y, axis=1) == 0)[0]
        self.idx_M       = np.where(np.argmax(y, axis=1) == 1)[0]
        self.half        = batch_size // 2
        self.aug = ImageDataGenerator(
            rotation_range=20,
            width_shift_range=0.10,
            height_shift_range=0.10,
            zoom_range=0.15,
            shear_range=0.10,
            horizontal_flip=True,
            fill_mode='nearest'
        )

    def __len__(self):
        return len(self.idx_M) // self.half

    def __getitem__(self, idx):
        # Sample equal numbers from each class
        batch_F = np.random.choice(self.idx_F, self.half, replace=True)
        batch_M = np.random.choice(self.idx_M, self.half, replace=False)
        batch_idx = np.concatenate([batch_F, batch_M])
        np.random.shuffle(batch_idx)

        X_batch = self.X[batch_idx].copy()
        y_batch = self.y[batch_idx].copy()

        if self.augment:
            for i in range(len(X_batch)):
                X_batch[i] = self.aug.random_transform(X_batch[i])

        return X_batch, y_batch

    def on_epoch_end(self):
        np.random.shuffle(self.idx_M)

train_gen = BalancedGenerator(X_train_c, y_train, batch_size=32, augment=True)
val_gen   = BalancedGenerator(X_val_c,   y_val,   batch_size=32, augment=False)

```

```

# Verify one batch
xb, yb = train_gen[0]
yb_enc = np.argmax(yb, axis=1)
print(f"\nBatch check - F: {np.sum(yb_enc==0)} | M: {np.sum(yb_enc==1)}")
print("Pipeline ready.")

```

Applying CLAHE enhancement...

```

CLAHE: 100%|██████████| 4202/4202 [00:02<00:00, 1922.12it/s]
CLAHE: 100%|██████████| 898/898 [00:00<00:00, 2118.95it/s]
CLAHE: 100%|██████████| 900/900 [00:00<00:00, 2022.67it/s]

```

Train - F: 861 | M: 3341

Batch check - F: 16 | M: 16

Pipeline ready.

In [12]: # CELL 7: EfficientNetB0 with Balanced Generator

```

def build_efficientnet(input_shape=(224, 224, 1), num_classes=2):
    inp = layers.Input(shape=input_shape)
    x = layers.Conv2D(3, (1,1), padding='same')(inp)
    x = layers.BatchNormalization()(x)

    base = EfficientNetB0(include_top=False, weights='imagenet',
                          input_shape=(224, 224, 3))
    base.trainable = False

    x = base(x, training=False)
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dense(256, activation='relu',
                    kernel_regularizer=regularizers.l2(1e-4))(x)
    x = layers.BatchNormalization()(x)
    x = layers.Dropout(0.5)(x)
    x = layers.Dense(128, activation='relu',
                    kernel_regularizer=regularizers.l2(1e-4))(x)
    x = layers.Dropout(0.3)(x)
    out = layers.Dense(num_classes, activation='softmax')(x)

    return models.Model(inp, out)

tf.keras.backend.clear_session()
eff_model = build_efficientnet()

# Phase 1 - head only
eff_model.compile(optimizer=Adam(1e-3),
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

print("Phase 1: Training head...")
h1 = eff_model.fit(
    train_gen,
    validation_data=(X_val_c, y_val),
    epochs=30,
    callbacks=[
        callbacks.EarlyStopping(monitor='val_accuracy', patience=8,

```

```

                                restore_best_weights=True, verbose=1),
callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5,
                             patience=4, min_lr=1e-7, verbose=1),
callbacks.ModelCheckpoint('model_efficientnet.h5', monitor='val_accuracy',
                           save_best_only=True, verbose=0)

],
verbose=1
)

# Phase 2 – fine-tune top 50 layers
print("\nPhase 2: Fine-tuning...")
for layer in eff_model.layers:
    if hasattr(layer, 'layers'):
        for l in layer.layers[-50:]:
            if not isinstance(l, layers.BatchNormalization):
                l.trainable = True

eff_model.compile(optimizer=Adam(2e-5),
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

h2 = eff_model.fit(
    train_gen,
    validation_data=(X_val_c, y_val),
    epochs=40,
    callbacks=[
        callbacks.EarlyStopping(monitor='val_accuracy', patience=10,
                                restore_best_weights=True, verbose=1),
        callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.3,
                                     patience=5, min_lr=1e-8, verbose=1),
        callbacks.ModelCheckpoint('model_efficientnet.h5', monitor='val_accuracy',
                                   save_best_only=True, verbose=0)
    ],
    verbose=1
)

loss, acc = eff_model.evaluate(X_test_c, y_test, verbose=0)
print(f"\nEfficientNetB0 – Test Accuracy : {acc*100:.2f}%")
print(f"EfficientNetB0 – Test Loss      : {loss:.4f}")

# Detailed report
y_pred = np.argmax(eff_model.predict(X_test_c, verbose=0), axis=1)
y_true = np.argmax(y_test, axis=1)
print("\nClassification Report:")
print(classification_report(y_true, y_pred, target_names=le.classes_))

```

Phase 1: Training head...

Epoch 1/30

208/208 ————— **0s** 113ms/step - accuracy: 0.5346 - loss: 0.8762

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

208/208 ————— **60s** 171ms/step - accuracy: 0.5346 - loss: 0.8759 -
val_accuracy: 0.2049 - val_loss: 0.9038 - learning_rate: 0.0010

Epoch 2/30

208/208 ————— **24s** 114ms/step - accuracy: 0.5625 - loss: 0.7607 -
val_accuracy: 0.2049 - val_loss: 0.9259 - learning_rate: 0.0010

Epoch 3/30

208/208 ————— **0s** 111ms/step - accuracy: 0.5779 - loss: 0.7238

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

208/208 ————— **24s** 117ms/step - accuracy: 0.5780 - loss: 0.7238 -
val_accuracy: 0.7205 - val_loss: 0.6053 - learning_rate: 0.0010

Epoch 4/30

208/208 ————— **0s** 111ms/step - accuracy: 0.5978 - loss: 0.7150

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

208/208 ————— **24s** 117ms/step - accuracy: 0.5978 - loss: 0.7149 -
val_accuracy: 0.7706 - val_loss: 0.5355 - learning_rate: 0.0010

Epoch 5/30

208/208 ————— **24s** 115ms/step - accuracy: 0.6069 - loss: 0.7003 -
val_accuracy: 0.6960 - val_loss: 0.6160 - learning_rate: 0.0010

Epoch 6/30

208/208 ————— **0s** 109ms/step - accuracy: 0.6033 - loss: 0.6990

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

208/208 ————— **24s** 114ms/step - accuracy: 0.6033 - loss: 0.6989 -
val_accuracy: 0.8040 - val_loss: 0.5357 - learning_rate: 0.0010
Epoch 7/30
208/208 ————— **24s** 113ms/step - accuracy: 0.6391 - loss: 0.6720 -
val_accuracy: 0.6058 - val_loss: 0.6982 - learning_rate: 0.0010
Epoch 8/30
208/208 ————— **0s** 110ms/step - accuracy: 0.6268 - loss: 0.6753
Epoch 8: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
208/208 ————— **24s** 113ms/step - accuracy: 0.6268 - loss: 0.6753 -
val_accuracy: 0.6659 - val_loss: 0.6546 - learning_rate: 0.0010
Epoch 9/30
208/208 ————— **24s** 113ms/step - accuracy: 0.6514 - loss: 0.6565 -
val_accuracy: 0.6682 - val_loss: 0.6543 - learning_rate: 5.0000e-04
Epoch 10/30
208/208 ————— **24s** 113ms/step - accuracy: 0.6284 - loss: 0.6677 -
val_accuracy: 0.6537 - val_loss: 0.6840 - learning_rate: 5.0000e-04
Epoch 11/30
208/208 ————— **24s** 115ms/step - accuracy: 0.6326 - loss: 0.6687 -
val_accuracy: 0.6648 - val_loss: 0.6649 - learning_rate: 5.0000e-04
Epoch 12/30
208/208 ————— **0s** 109ms/step - accuracy: 0.6404 - loss: 0.6597
Epoch 12: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
208/208 ————— **24s** 113ms/step - accuracy: 0.6404 - loss: 0.6597 -
val_accuracy: 0.7572 - val_loss: 0.5654 - learning_rate: 5.0000e-04
Epoch 13/30
208/208 ————— **24s** 113ms/step - accuracy: 0.6549 - loss: 0.6515 -
val_accuracy: 0.6927 - val_loss: 0.6371 - learning_rate: 2.5000e-04
Epoch 14/30
208/208 ————— **24s** 114ms/step - accuracy: 0.6577 - loss: 0.6517 -
val_accuracy: 0.6637 - val_loss: 0.6634 - learning_rate: 2.5000e-04
Epoch 14: early stopping
Restoring model weights from the end of the best epoch: 6.

Phase 2: Fine-tuning...

Epoch 1/40

2026-03-24 14:34:47.537948: E external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay kernel timed out: measured time has sub-optimal accuracy. There may be a missing warmup execution, please investigate in Nsight Systems.
2026-03-24 14:34:47.745273: E external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay kernel timed out: measured time has sub-optimal accuracy. There may be a missing warmup execution, please investigate in Nsight Systems.
2026-03-24 14:34:48.064834: E external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay kernel timed out: measured time has sub-optimal accuracy. There may be a missing warmup execution, please investigate in Nsight Systems.
2026-03-24 14:34:48.271942: E external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay kernel timed out: measured time has sub-optimal accuracy. There may be a missing warmup execution, please investigate in Nsight Systems.

208/208 ————— **0s** 112ms/step - accuracy: 0.6355 - loss: 0.6783

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

208/208 ————— **67s** 172ms/step - accuracy: 0.6354 - loss: 0.6783 -
val_accuracy: 0.7506 - val_loss: 0.6003 - learning_rate: 2.0000e-05

Epoch 2/40

208/208 ————— **0s** 114ms/step - accuracy: 0.6363 - loss: 0.6650

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

208/208 ————— **25s** 120ms/step - accuracy: 0.6363 - loss: 0.6650 -
val_accuracy: 0.7728 - val_loss: 0.5787 - learning_rate: 2.0000e-05

Epoch 3/40

208/208 ————— **24s** 114ms/step - accuracy: 0.6500 - loss: 0.6660 -
val_accuracy: 0.6682 - val_loss: 0.6730 - learning_rate: 2.0000e-05

Epoch 4/40

208/208 ————— **25s** 118ms/step - accuracy: 0.6415 - loss: 0.6605 -
val_accuracy: 0.7706 - val_loss: 0.5729 - learning_rate: 2.0000e-05

Epoch 5/40

208/208 ————— **0s** 115ms/step - accuracy: 0.6467 - loss: 0.6565

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

208/208 ————— 25s 122ms/step - accuracy: 0.6467 - loss: 0.6565 -
val_accuracy: 0.8040 - val_loss: 0.4907 - learning_rate: 2.0000e-05
Epoch 6/40

208/208 ————— 25s 118ms/step - accuracy: 0.6608 - loss: 0.6479 -
val_accuracy: 0.7094 - val_loss: 0.6195 - learning_rate: 2.0000e-05
Epoch 7/40

208/208 ————— 24s 117ms/step - accuracy: 0.6561 - loss: 0.6394 -
val_accuracy: 0.6693 - val_loss: 0.6758 - learning_rate: 2.0000e-05
Epoch 8/40

208/208 ————— 24s 115ms/step - accuracy: 0.6462 - loss: 0.6508 -
val_accuracy: 0.7127 - val_loss: 0.6116 - learning_rate: 2.0000e-05
Epoch 9/40

208/208 ————— 24s 114ms/step - accuracy: 0.6571 - loss: 0.6505 -
val_accuracy: 0.7617 - val_loss: 0.5586 - learning_rate: 2.0000e-05
Epoch 10/40

208/208 ————— 25s 120ms/step - accuracy: 0.6692 - loss: 0.6361 -
val_accuracy: 0.8018 - val_loss: 0.4822 - learning_rate: 2.0000e-05
Epoch 11/40

208/208 ————— 24s 117ms/step - accuracy: 0.6652 - loss: 0.6380 -
val_accuracy: 0.7038 - val_loss: 0.6197 - learning_rate: 2.0000e-05
Epoch 12/40

208/208 ————— 23s 113ms/step - accuracy: 0.6636 - loss: 0.6396 -
val_accuracy: 0.7283 - val_loss: 0.6020 - learning_rate: 2.0000e-05
Epoch 13/40

208/208 ————— 24s 116ms/step - accuracy: 0.6691 - loss: 0.6291 -
val_accuracy: 0.7550 - val_loss: 0.5508 - learning_rate: 2.0000e-05
Epoch 14/40

208/208 ————— 25s 121ms/step - accuracy: 0.6651 - loss: 0.6404 -
val_accuracy: 0.7706 - val_loss: 0.5429 - learning_rate: 2.0000e-05
Epoch 15/40

208/208 ————— 25s 122ms/step - accuracy: 0.6775 - loss: 0.6262 -
val_accuracy: 0.8040 - val_loss: 0.4687 - learning_rate: 2.0000e-05
Epoch 15: early stopping
Restoring model weights from the end of the best epoch: 5.

EfficientNetB0 – Test Accuracy : 80.44%
EfficientNetB0 – Test Loss : 0.4920

Classification Report:

	precision	recall	f1-score	support
F	0.80	0.06	0.12	185
M	0.80	1.00	0.89	715
accuracy			0.80	900
macro avg	0.80	0.53	0.51	900
weighted avg	0.80	0.80	0.73	900

In [14]: # CELL 8: Training with Augmentation + Fine-Tuning

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau, Model
from sklearn.utils.class_weight import compute_class_weight
```

```

import numpy as np

# --- Data Augmentation ---
train_datagen = ImageDataGenerator(
    rotation_range=10,
    zoom_range=0.1,
    width_shift_range=0.05,
    height_shift_range=0.05,
    horizontal_flip=True
)

val_datagen = ImageDataGenerator()

train_gen = train_datagen.flow(X_train, y_train, batch_size=32)
val_gen = val_datagen.flow(X_val, y_val, batch_size=32)

# --- Class Weights (IMPORTANT for imbalance) ---
y_train_labels = np.argmax(y_train, axis=1)
class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(y_train_labels),
    y=y_train_labels
)
class_weights = dict(enumerate(class_weights))
print("Class Weights:", class_weights)

# --- Callbacks ---
callbacks = [
    EarlyStopping(patience=5, restore_best_weights=True, monitor='val_loss'),
    ReduceLROnPlateau(factor=0.3, patience=3, min_lr=1e-6, verbose=1),
    ModelCheckpoint("best_model.h5", save_best_only=True, monitor='val_accuracy')
]

# --- Compile Model ---
eff_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# --- Initial Training ---
history = eff_model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=15,
    class_weight=class_weights,
    callbacks=callbacks
)

# =====
# ==> Fine-Tuning Step (KEY for 90%+ accuracy)
# =====

```

```

# Unfreeze top layers of EfficientNet
for layer in eff_model.layers[-50:]: # unfreeze last 50 layers
    layer.trainable = True

eff_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5), # lower LR
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history_finetune = eff_model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=10,
    class_weight=class_weights,
    callbacks=callbacks
)

```

Class Weights: {0: np.float64(2.440185830429733), 1: np.float64(0.6288536366357378)}

Epoch 1/15

132/132 ————— 0s 372ms/step - accuracy: 0.6669 - loss: 0.6487

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

132/132 ————— 131s 470ms/step - accuracy: 0.6670 - loss: 0.6486
- val_accuracy: 0.7060 - val_loss: 0.6181 - learning_rate: 1.0000e-04

Epoch 2/15

132/132 ————— 0s 120ms/step - accuracy: 0.7160 - loss: 0.6178

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

132/132 ————— 18s 132ms/step - accuracy: 0.7160 - loss: 0.6177 -
val_accuracy: 0.7684 - val_loss: 0.5410 - learning_rate: 1.0000e-04

Epoch 3/15

132/132 ————— 0s 122ms/step - accuracy: 0.7318 - loss: 0.5913

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

132/132 ————— **18s** 134ms/step - accuracy: 0.7319 - loss: 0.5912 -
val_accuracy: 0.7795 - val_loss: 0.5388 - learning_rate: 1.0000e-04
Epoch 4/15

132/132 ————— **17s** 128ms/step - accuracy: 0.7597 - loss: 0.5548 -
val_accuracy: 0.3040 - val_loss: 1.5476 - learning_rate: 1.0000e-04
Epoch 5/15

132/132 ————— **17s** 127ms/step - accuracy: 0.7628 - loss: 0.5230 -
val_accuracy: 0.7584 - val_loss: 0.5560 - learning_rate: 1.0000e-04
Epoch 6/15

132/132 ————— **0s** 120ms/step - accuracy: 0.7767 - loss: 0.4924
Epoch 6: ReduceLR0nPlateau reducing learning rate to 2.9999999242136255e-05.

132/132 ————— **17s** 126ms/step - accuracy: 0.7767 - loss: 0.4924 -
val_accuracy: 0.6927 - val_loss: 0.6471 - learning_rate: 1.0000e-04
Epoch 7/15

132/132 ————— **0s** 118ms/step - accuracy: 0.8075 - loss: 0.4583

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

132/132 ————— **17s** 129ms/step - accuracy: 0.8075 - loss: 0.4583 -
val_accuracy: 0.8318 - val_loss: 0.4526 - learning_rate: 3.0000e-05
Epoch 8/15

132/132 ————— **0s** 116ms/step - accuracy: 0.7904 - loss: 0.4345

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

132/132 ————— **17s** 128ms/step - accuracy: 0.7906 - loss: 0.4346 -
val_accuracy: 0.8352 - val_loss: 0.4410 - learning_rate: 3.0000e-05
Epoch 9/15

132/132 ————— **0s** 117ms/step - accuracy: 0.8233 - loss: 0.4223

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

```

132/132 ————— 17s 129ms/step - accuracy: 0.8233 - loss: 0.4222 -
val_accuracy: 0.8408 - val_loss: 0.4364 - learning_rate: 3.0000e-05
Epoch 10/15
132/132 ————— 16s 123ms/step - accuracy: 0.8373 - loss: 0.4127 -
val_accuracy: 0.8073 - val_loss: 0.4657 - learning_rate: 3.0000e-05
Epoch 11/15
132/132 ————— 17s 125ms/step - accuracy: 0.8261 - loss: 0.3888 -
val_accuracy: 0.8040 - val_loss: 0.4737 - learning_rate: 3.0000e-05
Epoch 12/15
132/132 ————— 0s 117ms/step - accuracy: 0.8475 - loss: 0.3679
Epoch 12: ReduceLR0nPlateau reducing learning rate to 8.999999772640877e-06.
132/132 ————— 16s 123ms/step - accuracy: 0.8475 - loss: 0.3680 -
val_accuracy: 0.8207 - val_loss: 0.5096 - learning_rate: 3.0000e-05
Epoch 13/15
132/132 ————— 16s 123ms/step - accuracy: 0.8465 - loss: 0.3476 -
val_accuracy: 0.8185 - val_loss: 0.4584 - learning_rate: 9.0000e-06
Epoch 14/15
132/132 ————— 17s 125ms/step - accuracy: 0.8654 - loss: 0.3427 -
val_accuracy: 0.8062 - val_loss: 0.4706 - learning_rate: 9.0000e-06
Epoch 1/10
132/132 ————— 129s 456ms/step - accuracy: 0.8243 - loss: 0.4078
- val_accuracy: 0.7940 - val_loss: 0.4956 - learning_rate: 1.0000e-05
Epoch 2/10
132/132 ————— 16s 124ms/step - accuracy: 0.8469 - loss: 0.3915 -
val_accuracy: 0.8051 - val_loss: 0.4811 - learning_rate: 1.0000e-05
Epoch 3/10
132/132 ————— 16s 120ms/step - accuracy: 0.8330 - loss: 0.4024 -
val_accuracy: 0.7973 - val_loss: 0.5127 - learning_rate: 1.0000e-05
Epoch 4/10
132/132 ————— 16s 122ms/step - accuracy: 0.8521 - loss: 0.3703 -
val_accuracy: 0.7873 - val_loss: 0.5111 - learning_rate: 1.0000e-05
Epoch 5/10
132/132 ————— 0s 112ms/step - accuracy: 0.8521 - loss: 0.3728
Epoch 5: ReduceLR0nPlateau reducing learning rate to 2.9999999242136253e-06.
132/132 ————— 16s 117ms/step - accuracy: 0.8520 - loss: 0.3728 -
val_accuracy: 0.7840 - val_loss: 0.5095 - learning_rate: 1.0000e-05
Epoch 6/10
132/132 ————— 16s 122ms/step - accuracy: 0.8486 - loss: 0.3735 -
val_accuracy: 0.7639 - val_loss: 0.5493 - learning_rate: 3.0000e-06
Epoch 7/10
132/132 ————— 16s 121ms/step - accuracy: 0.8590 - loss: 0.3625 -
val_accuracy: 0.7706 - val_loss: 0.5467 - learning_rate: 3.0000e-06

```

In [15]: *# CELL 9: Improve Generalization (Reduce Overfitting)*

```

from tensorflow.keras.layers import Dropout
from tensorflow.keras.models import Model

# --- Add Dropout to your model (if not already present) ---
x = eff_model.layers[-2].output # take second last layer
x = Dropout(0.5)(x)             # strong dropout
new_output = tf.keras.layers.Dense(y_train.shape[1], activation='softmax')(x)

eff_model = Model(inputs=eff_model.input, outputs=new_output)

```

```

# --- Freeze MOST layers again ---
for layer in eff_model.layers[:-20]: # train only last 20 layers
    layer.trainable = False

# --- Compile ---
eff_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# --- Stronger Augmentation ---
train_datagen = ImageDataGenerator(
    rotation_range=15,
    zoom_range=0.2,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True,
    fill_mode='nearest'
)

train_gen = train_datagen.flow(X_train, y_train, batch_size=32)
val_gen = ImageDataGenerator().flow(X_val, y_val, batch_size=32)

# --- Train again ---
history = eff_model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=10,
    callbacks=callbacks
)

```

Epoch 1/10
132/132 ————— **131s** 472ms/step - accuracy: 0.6216 - loss: 0.8179
- val_accuracy: 0.2160 - val_loss: 2.3220 - learning_rate: 1.0000e-04

Epoch 2/10
132/132 ————— **17s** 130ms/step - accuracy: 0.7746 - loss: 0.5244 -
val_accuracy: 0.2261 - val_loss: 2.3616 - learning_rate: 1.0000e-04

Epoch 3/10
132/132 ————— **18s** 137ms/step - accuracy: 0.7984 - loss: 0.4670 -
val_accuracy: 0.7840 - val_loss: 0.5519 - learning_rate: 1.0000e-04

Epoch 4/10
132/132 ————— **17s** 131ms/step - accuracy: 0.8314 - loss: 0.3983 -
val_accuracy: 0.8029 - val_loss: 0.6359 - learning_rate: 1.0000e-04

Epoch 5/10
132/132 ————— **17s** 127ms/step - accuracy: 0.8381 - loss: 0.3966 -
val_accuracy: 0.7606 - val_loss: 0.5888 - learning_rate: 1.0000e-04

Epoch 6/10
132/132 ————— **0s** 118ms/step - accuracy: 0.8532 - loss: 0.3709

Epoch 6: ReduceLR0nPlateau reducing learning rate to 2.9999999242136255e-05.
132/132 ————— **16s** 124ms/step - accuracy: 0.8532 - loss: 0.3710 -
val_accuracy: 0.7918 - val_loss: 0.6383 - learning_rate: 1.0000e-04

Epoch 7/10
132/132 ————— **17s** 126ms/step - accuracy: 0.8680 - loss: 0.3492 -
val_accuracy: 0.8107 - val_loss: 0.6293 - learning_rate: 3.0000e-05

Epoch 8/10
132/132 ————— **17s** 126ms/step - accuracy: 0.8614 - loss: 0.3672 -
val_accuracy: 0.8029 - val_loss: 0.6634 - learning_rate: 3.0000e-05

```
In [20]: callbacks = [
    EarlyStopping(patience=6, restore_best_weights=True, monitor='val_loss'),

    ReduceLR0nPlateau(
        factor=0.3,
        patience=2,
        min_lr=1e-6,
        verbose=1
    ),

    ModelCheckpoint(
        "best_model.keras",
        monitor='val_loss',
        save_best_only=True,
        verbose=1
    )
]
loss_fn = tf.keras.losses.CategoricalCrossentropy(label_smoothing=0.1)

eff_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=5e-5),
    loss=loss_fn,
    metrics=['accuracy']
)

history = eff_model.fit(
    train_gen,
```

```
validation_data=val_gen,  
epochs=10,  
callbacks=callbacks  
)  
print("Done")
```


Epoch 1/10
132/132 ————— **0s** 361ms/step - accuracy: 0.8231 - loss: 0.5507
Epoch 1: val_loss improved from inf to 0.63214, saving model to best_model.keras

132/132 ————— **132s** 479ms/step - accuracy: 0.8232 - loss: 0.5506
- val_accuracy: 0.8207 - val_loss: 0.6321 - learning_rate: 5.0000e-05

Epoch 2/10
132/132 ————— **0s** 119ms/step - accuracy: 0.8274 - loss: 0.5249
Epoch 2: val_loss did not improve from 0.63214

132/132 ————— **17s** 125ms/step - accuracy: 0.8275 - loss: 0.5248 -
val_accuracy: 0.7895 - val_loss: 0.7008 - learning_rate: 5.0000e-05

Epoch 3/10
132/132 ————— **0s** 117ms/step - accuracy: 0.8379 - loss: 0.5086
Epoch 3: val_loss improved from 0.63214 to 0.60333, saving model to best_model.keras

132/132 ————— **17s** 131ms/step - accuracy: 0.8379 - loss: 0.5085 -
val_accuracy: 0.8341 - val_loss: 0.6033 - learning_rate: 5.0000e-05

Epoch 4/10
132/132 ————— **0s** 122ms/step - accuracy: 0.8444 - loss: 0.4753
Epoch 4: val_loss did not improve from 0.60333

132/132 ————— **17s** 128ms/step - accuracy: 0.8443 - loss: 0.4754 -
val_accuracy: 0.8029 - val_loss: 0.7232 - learning_rate: 5.0000e-05

Epoch 5/10
132/132 ————— **0s** 122ms/step - accuracy: 0.8534 - loss: 0.4743
Epoch 5: val_loss improved from 0.60333 to 0.52463, saving model to best_model.keras

132/132 ————— **18s** 136ms/step - accuracy: 0.8534 - loss: 0.4743 -
val_accuracy: 0.8396 - val_loss: 0.5246 - learning_rate: 5.0000e-05

Epoch 6/10
132/132 ————— **0s** 121ms/step - accuracy: 0.8568 - loss: 0.4637
Epoch 6: val_loss did not improve from 0.52463

132/132 ————— **17s** 127ms/step - accuracy: 0.8568 - loss: 0.4637 -
val_accuracy: 0.8229 - val_loss: 0.5317 - learning_rate: 5.0000e-05

Epoch 7/10
132/132 ————— **0s** 117ms/step - accuracy: 0.8674 - loss: 0.4528
Epoch 7: ReduceLROnPlateau reducing learning rate to 1.4999999621068127e-05.

Epoch 7: val_loss did not improve from 0.52463

132/132 ————— **16s** 123ms/step - accuracy: 0.8673 - loss: 0.4529 -
val_accuracy: 0.8307 - val_loss: 0.5639 - learning_rate: 5.0000e-05

Epoch 8/10
132/132 ————— **0s** 116ms/step - accuracy: 0.8589 - loss: 0.4598
Epoch 8: val_loss did not improve from 0.52463

132/132 ————— **16s** 122ms/step - accuracy: 0.8589 - loss: 0.4598 -
val_accuracy: 0.8408 - val_loss: 0.5393 - learning_rate: 1.5000e-05

Epoch 9/10
132/132 ————— **0s** 121ms/step - accuracy: 0.8710 - loss: 0.4449
Epoch 9: val_loss improved from 0.52463 to 0.52349, saving model to best_model.keras

132/132 ————— **18s** 135ms/step - accuracy: 0.8709 - loss: 0.4450 -
val_accuracy: 0.8363 - val_loss: 0.5235 - learning_rate: 1.5000e-05

Epoch 10/10
132/132 ————— **0s** 119ms/step - accuracy: 0.8799 - loss: 0.4385
Epoch 10: val_loss did not improve from 0.52349

132/132 ————— 17s 124ms/step - accuracy: 0.8799 - loss: 0.4385 -
val_accuracy: 0.8408 - val_loss: 0.5244 - learning_rate: 1.5000e-05
Done

In [21]: # CELL 11: Test Time Augmentation (TTA)

```
import numpy as np

def tta_predict(model, X, n_aug=5):
    preds = []

    for i in range(n_aug):
        aug = ImageDataGenerator(
            rotation_range=10,
            zoom_range=0.1,
            width_shift_range=0.05,
            height_shift_range=0.05,
            horizontal_flip=True
        )

        gen = aug.flow(X, batch_size=32, shuffle=False)
        pred = model.predict(gen, verbose=0)
        preds.append(pred)

    return np.mean(preds, axis=0)

# --- Run TTA ---
tta_preds = tta_predict(eff_model, X_val)

# --- Convert to labels ---
y_pred = np.argmax(tta_preds, axis=1)
y_true = np.argmax(y_val, axis=1)

# --- Accuracy ---
from sklearn.metrics import accuracy_score
print("TTA Accuracy:", accuracy_score(y_true, y_pred))
```

TTA Accuracy: 0.8429844097995546

In [22]: # CELL 12: Apply CLAHE during training (GAME CHANGER)

```
import cv2

def apply_clahe_batch(X):
    X_clahe = []
    for img in X:
        img = (img * 255).astype('uint8') # convert back to uint8
        if len(img.shape) == 3:
            img = img[:, :, 0]

        clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
        img = clahe.apply(img)

    img = img / 255.0
```

```
        img = np.expand_dims(img, axis=-1)
        X_clahe.append(img)

    return np.array(X_clahe)

# Apply to datasets
X_train_clahe = apply_clahe_batch(X_train)
X_val_clahe   = apply_clahe_batch(X_val)

# Train again
history = eff_model.fit(
    train_datagen.flow(X_train_clahe, y_train, batch_size=32),
    validation_data=(X_val_clahe, y_val),
    epochs=8,
    callbacks=callbacks
)
```

```

Epoch 1/8
132/132 ————— 0s 119ms/step - accuracy: 0.8611 - loss: 0.4702
Epoch 1: val_loss did not improve from 0.2349
132/132 ————— 28s 209ms/step - accuracy: 0.8611 - loss: 0.4701
- val_accuracy: 0.8196 - val_loss: 0.5647 - learning_rate: 1.5000e-05
Epoch 2/8
132/132 ————— 0s 118ms/step - accuracy: 0.8617 - loss: 0.4732
Epoch 2: val_loss did not improve from 0.2349
132/132 ————— 17s 125ms/step - accuracy: 0.8617 - loss: 0.4732
- val_accuracy: 0.8316 - val_loss: 0.5563 - learning_rate: 1.5000e-05
Epoch 3/8
132/132 ————— 0s 118ms/step - accuracy: 0.8500 - loss: 0.2600
Epoch 3: val_loss did not improve from 0.2349
132/132 ————— 17s 123ms/step - accuracy: 0.8509 - loss: 0.2607
- val_accuracy: 0.8310 - val_loss: 0.5470 - learning_rate: 1.5000e-05
Epoch 4/8
132/132 ————— 0s 114ms/step - accuracy: 0.0727 - loss: 0.3447
Epoch 4: val_loss did not improve from 0.2349
132/132 ————— 10s 123ms/step - accuracy: 0.0726 - loss: 0.4448
- val_accuracy: 0.0352 - val_loss: 0.5410 - learning_rate: 1.5000e-05
Epoch 5/8
132/132 ————— 0s 118ms/step - accuracy: 0.0614 - loss: 0.4457
Epoch 5: val_loss did not improve from 0.2349
132/132 ————— 10s 124ms/step - accuracy: 0.0614 - loss: 0.4457
- val_accuracy: 0.0261 - val_loss: 0.5567 - learning_rate: 1.5000e-05
Epoch 6/8
132/132 ————— 0s 118ms/step - accuracy: 0.0757 - loss: 0.4470
Epoch 6: ReduceLROnPlateau reducing learning rate to 4.490000606320430e-06.

Epoch 6: val_loss did not improve from 0.2349
132/132 ————— 16s 124ms/step - accuracy: 0.0756 - loss: 0.2478
- val_accuracy: 0.0103 - val_loss: 0.6042 - learning_rate: 1.5000e-05
Epoch 7/8
132/132 ————— 0s 112ms/step - accuracy: 0.0727 - loss: 0.3605
Epoch 7: val_loss did not improve from 0.2349
132/132 ————— 16s 119ms/step - accuracy: 0.0728 - loss: 0.4604
- val_accuracy: 0.0001 - val_loss: 0.2349 - learning_rate: 4.5000e-06
Epoch 8/8
132/132 ————— 0s 119ms/step - accuracy: 0.9013 - loss: 0.2420
Epoch 8: val_loss improved from 0.2349 to 0.21439, saving model to best_model.keras
132/132 ————— 17s 131ms/step - accuracy: 0.9013 - loss: 0.2420
- val_accuracy: 0.9001 - val_loss: 0.2143 - learning_rate: 4.5000e-06

```